



AUDIT REPORT

July 2026

For



unicef 
for every child



Table of Content

Executive Summary	04
Number of Security Issues per Severity	05
Summary of Issues	06
Checked Vulnerabilities	07
Techniques and Methods	09
Types of Severity	11
Types of Issues	12
Severity Matrix	13
 High Severity Issues	14
1. Permissionless buyNft lets anyone claim any schoolId, poison metadata, and permanently block legitimate minting	14
 Low Severity Issues	15
2. Missing zero-address validation on donationReceiver	15
3. baseFee has no positivity guard, enabling free permissionless minting when set to zero	16
4. Escrow.onERC721Received accepts any ERC721 and emits spoofable receipt events	17
 Informational Severity Issues	18
5. safeMint return signature mismatch between INft and Nft	18
6. Escrow.withdraw is not marked nonReentrant	19
Functional Tests	20
Formal Verification	23
Threat Model	26
Automated Tests	39
Closing Summary & Disclaimer	40



Executive Summary

Project Name	GigaBlocks
Protocol Type	NFT, Escrow
Project URL	https://giga.global/
Overview	The reviewed system mints on-chain School NFTs for the Giga initiative. GigaMinter is the minting hub: the admin can mint School NFTs to a chosen recipient, and donors can mint by paying a base fee, with the received funds forwarded to a donation receiver. A minted NFT may be delivered directly to a recipient, or routed through the Escrow contract, which holds a School NFT in custody until the owner releases it to the destination school wallet.
Audit Scope	The scope of this audit was to analyze the in-scope Gigablocks smart contracts, GigaMinter and Escrow, for quality, security, and correctness.
Source Code link	https://github.com/giga-nft2-0/Giga_NFT_2.0-contracts
Branch	main
Contracts in Scope	src/contracts/GigaMinter.sol src/contracts/Escrow.sol
Commit Hash	2decd2b
Language	Solidity
Blockchain	EVM
Method	Manual Analysis, Threat Modelling
Review 1	9th July, 2026
Updated Code Received	9th July, 2026
Review 2	9th July, 2026
Fixed In	2f97cd0



Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0 (0%)
High	1 (16.67%)
Medium	0 (0%)
Low	3 (50%)
Informational	2 (33.33%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	1	0
	Partially Resolved	0	0	0	0	0
	Resolved	0	1	0	2	2



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Permissionless buyNft lets anyone claim any schoolId, poison metadata, and permanently block legitimate minting	High	Resolved
2	Missing zero-address validation on donationReceiver	Low	Resolved
3	baseFee has no positivity guard, enabling free permissionless minting when set to zero	Low	Resolved
4	Escrow.onERC721Received accepts any ERC721 and emits spoofable receipt events	Low	Acknowledged
5	safeMint return signature mismatch between INft and Nft	Informational	Resolved
6	Escrow.withdraw is not marked nonReentrant	Informational	Resolved



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ **Missing Zero Address Validation**

✓ **Private modifier**

✓ **Revert/require functions**

✓ **Multiple Sends**

✓ **Using suicide**

✓ **Using delegatecall**

✓ **Upgradeable safety**

✓ **Using throw**

✓ **Using inline assembly**

✓ **Style guide violation**

✓ **Unsafe type inference**

✓ **Implicit visibility level**

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Manual analysis, threat modelling, functional testing.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



High Severity Issues

Permissionless buyNft lets anyone claim any schoolId, poison metadata, and permanently block legitimate minting

Resolved

Path

GigaMinter.sol

Function Name

`buyNft()`

Description

buyNft is externally callable by anyone, yet the caller fully controls the schoolId, the NFT recipient, and the 9 metadata values, all of which are forwarded unchanged to schoolNft.safeMint.

```
require(msg.value >= baseFee, "GigaMinter: Insufficient funds");
uint256 schoolNftTokenId = _mintSchools(_schoolId, _schoolNftTo, _values);
```

Because NftContent.generateTokenId binds a schoolId to a tokenId on a first-come, first-served basis and there is no burn or reset anywhere, an attacker can call buyNft("<real-school-id>", attacker, arbitraryValues) to permanently register that schoolId to a token they own with metadata they choose. A later admin mintNft for the same schoolId reverts inside _safeMint because the token already exists, so the real school can never receive its canonical NFT.

Impact

An attacker permanently squats arbitrary school identifiers, mints NFTs that render via tokenURI as the official Giga NFT for those schools with attacker chosen data, and permanently denies the admin the ability to mint the affected schools with no recovery path.

Recommendation

It should not let a donor define a real school's identifier or metadata. Or gate buyNft behind a whitelist so that only pre-approved addresses can call it. Additionally add an admin reset/burn capability so a poisoned schoolId can be reclaimed.



Low Severity Issues

Missing zero-address validation on donationReceiver

Resolved

Path

GigaMinter.sol

Function Name

`constructor()`, `setDonationReceiver()`

Description

Neither the constructor nor `setDonationReceiver` validates that `donationReceiver` is non-zero. If `donationReceiver` is set to `address(0)`, every subsequent `buyNft` still succeeds but the `.call` in `_transferDonation` sends the paid ETH to the zero address, effectively burning donor funds until the owner corrects the configuration.

Recommendation

Add `require(_newDonationReceiver != address(0), "GigaMinter: zero address")` to both the constructor and `setDonationReceiver`.



Low Severity Issues

baseFee has no positivity guard, enabling free permissionless minting when set to zero

Resolved

Path

GigaMinter.sol

Function Name

`constructor()` , `setBaseFee()` , `buyNft()`

Description

Neither the constructor nor `setBaseFee` enforces that `baseFee` is greater than zero, and `buyNft` only checks `msg.value >= baseFee`.

```
require(msg.value >= baseFee, "GigaMinter: Insufficient funds");
```

If `baseFee` is zero, by configuration or if left unset, the check passes for `msg.value == 0`, so any address can mint school NFTs at no cost through the permissionless `buyNft` path.

Recommendation

If a non-zero fee is always required, add `require(_newBaseFee > 0, "GigaMinter: zero base fee")`.



Low Severity Issues

Escrow.onERC721Received accepts any ERC721 and emits spoofable receipt events

Acknowledged

Path

Escrow.sol

Function Name

`onERC721Received()`

Description

`onERC721Received` is public, returns the ERC721 receiver selector unconditionally, and never checks that `msg.sender` equals the configured `nftAddress`. Any account can call it directly with arbitrary operator, from, and tokenId values to emit a forged `ERC721Received` event without moving any token. Off-chain systems that treat this event as a genuine deposit can be misled.

Recommendation

Add a modifier so only `nftAddress` can mint



Informational Severity Issues

safeMint return signature mismatch between INft and Nft

Resolved

Path

GigaMinter.sol

Function Name

`_mintSchools()`

Description

INft.safeMint is declared to return a single uint256, but the deployed Nft.safeMint returns two values (uint256 tokenId, bytes32 tokenHash), and _mintSchools decodes only the first word.

```
uint256 schoolNftTokenId = schoolNft.safeMint(_schoolId, _schoolNftTo, _values);
```

The call works because ABI decoding reads only the first return value, but the interface no longer matches the implementation, which is error prone for future integrators reading the interface.

Recommendation

Align INft.safeMint with the implementation by returning both values, or intentionally document that the tokenHash return value is discarded by callers.



Informational Severity Issues

Escrow.withdraw is not marked nonReentrant

Resolved

Path

Escrow.sol

Function Name

`withdraw()`

Description

withdraw performs a raw ETH .call to an owner-supplied address but, unlike transferSchoolNft and bulkTransferSchoolNft, is not marked nonReentrant.

```
(bool sent, ) = _to.call{value: _amount}("");  
require(sent, "Failed to send Ether");
```

There is no exploitable re-entrancy path today because the function is owner-only and the contract holds no per user accounting, but the inconsistency with the other value moving functions is worth aligning.

Recommendation

Add the nonReentrant modifier to withdraw for consistency with the other value-moving functions.



Functional Tests

GigaMinter

- ✓ constructor sets the owner to the provided gigaAdmin via Ownable
- ✓ constructor stores the schoolNft contract address
- ✓ constructor stores the baseFee value
- ✓ constructor stores the donationReceiver as a payable address
- ✓ constructor sets isBuyingPaused to false at deployment
- ✓ onlyOwner restricts mintNft to the owner
- ✓ onlyOwner restricts batchMintNft to the owner
- ✓ onlyOwner restricts setBaseFee to the owner
- ✓ onlyOwner restricts setDonationReceiver to the owner
- ✓ onlyOwner restricts updateBuyingStatus to the owner
- ✓ whenBuyingNotPaused blocks buyNft while isBuyingPaused is true
- ✓ nonReentrant guards mintNft, buyNft and batchMintNft against reentrancy
- ✓ mintNft reverts for any non-owner caller
- ✓ mintNft mints one school NFT to the given recipient via _mintSchools
- ✓ mintNft emits SchoolNFTMinted with donor address(0) and zero donation
- ✓ mintNft reverts if the schoolId was already minted
- ✓ buyNft is callable by any address without a role
- ✓ buyNft reverts if msg.value is below baseFee
- ✓ buyNft reverts while buying is paused
- ✓ buyNft mints one school NFT to the caller specified recipient
- ✓ buyNft forwards the entire msg.value to donationReceiver
- ✓ buyNft emits SchoolNFTMinted with the caller as donor and msg.value as amount



- ✓ `buyNft` reverts if the `schoolId` was already minted
- ✓ `batchMintNft` reverts for any non-owner caller
- ✓ `batchMintNft` reverts if `schoolIds` and `schoolNftTos` lengths differ
- ✓ `batchMintNft` reverts if `schoolIds` and `values` lengths differ
- ✓ `batchMintNft` mints one NFT per entry to each matching recipient
- ✓ `batchMintNft` emits `SchoolNFTMinted` per token with donor address(0)
- ✓ `batchMintNft` emits `BulkNftMinted` with the number of schools minted
- ✓ `batchMintNft` reverts if any `schoolId` in the batch was already minted
- ✓ `setBaseFee` reverts for non-owner callers
- ✓ `setBaseFee` updates `baseFee` to the new value
- ✓ `setDonationReceiver` reverts for non-owner callers
- ✓ `setDonationReceiver` updates `donationReceiver` to the new address
- ✓ `_transferDonation` increases `totalDonations` by the amount above the base fee
- ✓ `_transferDonation` increases `totalBaseFeeCollected` by the base fee
- ✓ `_transferDonation` reverts if the ETH transfer to `donationReceiver` fails
- ✓ `_transferDonation` emits `DonationTransferred` with the paid amount
- ✓ `updateBuyingStatus` reverts for non-owner callers
- ✓ `updateBuyingStatus` sets `isBuyingPaused` to the provided value
- ✓ `updateBuyingStatus` emits `BuyingStatusUpdated` with the new status

Escrow

- ✓ constructor sets the owner to the provided `gigaAdmin` via `Ownable`
- ✓ constructor stores the nft contract address in `nftAddress`
- ✓ `onERC721Received` returns the `IERC721Receiver` selector so safe transfers succeed
- ✓ `onERC721Received` emits `ERC721Received` with operator, from, tokenId and data
- ✓ `withdraw` reverts for non-owner callers



- ✓ `withdraw` sends the requested ETH amount to the recipient
- ✓ `withdraw` reverts if the ETH transfer fails
- ✓ `transferSchoolNft` reverts for non-owner callers
- ✓ `transferSchoolNft` transfers the school NFT from the escrow to the recipient
- ✓ `transferSchoolNft` emits `SchoolNftTransferred` with recipient and tokenId
- ✓ `transferSchoolNft` is guarded by `nonReentrant` against the receiver callback
- ✓ `bulkTransferSchoolNft` reverts for non-owner callers
- ✓ `bulkTransferSchoolNft` reverts if `tos` and `tokenIds` lengths differ
- ✓ `bulkTransferSchoolNft` transfers each NFT to its matching recipient
- ✓ `bulkTransferSchoolNft` emits `SchoolNftTransferred` for each transfer
- ✓ `bulkTransferSchoolNft` is guarded by `nonReentrant`
- ✓ `receive` accepts ETH from any sender and emits `Deposit`
- ✓ `transferERC721` reverts for non-owner callers
- ✓ `transferERC721` moves an ERC721 token from a given from address to a recipient
- ✓ `approveERC721` reverts for non-owner callers
- ✓ `approveERC721ForAll` reverts for non-owner callers
- ✓ ownership transfer requires the new owner to accept via `Ownable2Step`
- ✓ `nftAddress` getter returns the configured school NFT address



Formal Verification

GigaMinter

- ✓ rule_access_mintNft_onlyOwner - non-owner call to mintNft reverts
- ✓ rule_access_batchMintNft_onlyOwner - non-owner call to batchMintNft reverts
- ✓ rule_access_baseFee_onlyOwnerSetter - non-owner call to setBaseFee reverts
- ✓ rule_access_donationReceiver_onlyOwnerSetter - non-owner call to setDonationReceiver reverts
- ✓ rule_access_pause_onlyOwnerSetter - non-owner call to updateBuyingStatus reverts
- ✓ rule_access_ownership_onlyOwner - non-owner call to transferOwnership reverts
- ✓ rule_access_renounceOwnership_onlyOwner - non-owner call to renounceOwnership reverts
- ✓ rule_own_acceptOwnership_onlyPending - only the pendingOwner can accept ownership
- ✓ rule_access_buyNft_noEscalation - buyNft grants the caller no privilege escalation
- ✓ rule_pause_buyNftRevertsWhenPaused - buyNft reverts while isBuyingPaused is true
- ✓ rule_pause_setterSetsExactValue - updateBuyingStatus sets isBuyingPaused to the exact value
- ✓ rule_pause_adminMintNotGated - admin mint stays reachable while buying is paused
- ✓ rule_buyNft_revertsWhenUnderfunded - buyNft reverts when msg.value is below baseFee
- ✓ rule_conservation_buyNft_splitExact - buyNft splits the payment into base fee and donation exactly
- ✓ rule_conservation_buyNft_deltaSumEqualsValue - the increase in the two totals equals msg.value on buyNft
- ✓ rule_value_buyNft_noHold - GigaMinter holds no ETH after buyNft (all forwarded)
- ✓ rule_value_onlyBuyNftAddsEth - only buyNft moves ETH through the contract
- ✓ rule_mono_totalsNeverDecrease - totalDonations and totalBaseFeeCollected never decrease



- ✓ rule_state_totals_onlyBuyNft - only buyNft mutates the two totals
- ✓ rule_state_schoolNft_immutable - the schoolNft address never changes (no setter)
- ✓ rule_state_owner_onlyAcceptOrRenounce - owner changes only via acceptOwnership or renounce
- ✓ rule_state_pendingOwner_sinks - pendingOwner changes only through the two-step flow
- ✓ inv_totals_equal_forwarded - totals equal the forwarded amount for all direct methods

Escrow

- ✓ rule_access_withdraw_onlyOwner - non-owner call to withdraw reverts
- ✓ rule_access_transferSchoolNft_onlyOwner - non-owner call to transferSchoolNft reverts
- ✓ rule_access_bulkTransferSchoolNft_onlyOwner - non-owner call to bulkTransferSchoolNft reverts
- ✓ rule_access_transferERC721_onlyOwner - non-owner call to transferERC721 reverts
- ✓ rule_access_approveERC721_onlyOwner - non-owner call to approveERC721 reverts
- ✓ rule_access_approveERC721ForAll_onlyOwner - non-owner call to approveERC721ForAll reverts
- ✓ rule_access_ownership_onlyOwner_escrow - non-owner call to transferOwnership reverts
- ✓ rule_access_renounceOwnership_onlyOwner_escrow - non-owner call to renounceOwnership reverts
- ✓ rule_own_acceptOwnership_onlyPending_escrow - only the pendingOwner can accept ownership
- ✓ rule_access_onERC721Received_noEscalation - onERC721Received grants the caller no privilege escalation
- ✓ rule_access_receive_noEscalation - receive() grants the caller no privilege escalation
- ✓ rule_isolation_ethDecreaseOnlyOwnerWithdraw - only owner withdraw can decrease the Escrow ETH balance
- ✓ rule_value_withdrawExactDebit - withdraw debits exactly the requested amount



- ✓ **rule_value_withdrawCannotOverdraw** - withdraw cannot overdraw the Escrow balance
- ✓ **rule_value_ethIncreaseOnlyViaReceive** - ETH enters the Escrow only via receive()
- ✓ **rule_value_receiveAddsExact** - receive() increases the balance by exactly msg.value
- ✓ **rule_state_nftAddress_immutable** - nftAddress never changes (no setter)
- ✓ **rule_state_owner_onlyAcceptOrRenounce_escrow** - owner changes only via acceptOwnership or renounce
- ✓ **rule_state_pendingOwner_sinks_escrow** - pendingOwner changes only through the two-step flow
- ✓ **rule_own_renounceLocksAssets** - after renounce, withdraw and NFT transfer revert for everyone



Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
GigaAdmin (GigaMinter owner)	Admin / identity	Owner of GigaMinter; authorizes mintNft, batchMintNft, setBaseFee, setDonationReceiver, and updateBuyingStatus.	The GigaAdmin key remains honest, available, and correctly rotated.	Compromise lets the admin pause buying, redirect donations, change the fee, or mint arbitrary schools.
Escrow owner (Safe multisig)	Admin / identity	Owner of Escrow; authorizes withdraw, transferSchoolNft, bulkTransferSchoolNft, and the inherited arbitrary ERC721 transfer/approval helpers.	The multisig remains honest and available; delivery of NFTs to schools is at its discretion.	Can move any custodied School NFT or ETH; can move ERC721s from any address that approved the escrow; renouncing ownership permanently bricks custody.



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
donationReceiver	Mutable protocol configuration	Receives the entire msg.value forwarded by buyNft (base fee + donation).	Set to the protocol Safe and never the zero address.	A zero address burns donor ETH on every buyNft; a non-protocol address would also leak the base-fee revenue.
baseFee	Mutable protocol configuration	Read by buyNft to gate the minimum payment.	Intentionally set; non-zero when a fee is required.	A zero baseFee turns buyNft into a permissionless free mint.
schoolId and metadata (_values)	User-supplied input	Passed to mintNft/buyNft and forwarded unchanged to the NFT layer as the school identifier and its 9 metadata fields.	The caller supplies a correct, real schoolId and accurate metadata.	Because buyNft is permissionless, anyone can supply any schoolId and metadata, enabling identifier squatting and metadata poisoning.
ERC721 safe-transfer callbacks	Onchain callback	Escrow.onERC721Received is invoked by any ERC721 during safeTransferFrom; recipient onERC721Received fires during minting/delivery.	The callback is benign and the caller is the intended token contract.	onERC721Received accepts any collection and can be called directly to emit spoofed receipt events; recipient callbacks are re-entrancy vectors (mitigated by nonReentrant).

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Off-chain event consumers	Offchain / indexing	Index SchoolNFTMinted, DonationTransferred, ERC721Received, and SchoolNftTransferred logs.	Consumers validate transaction provenance and do not trust logs by event signature alone.	The public onERC721Received emitter is spoofable; event-only systems can mistake forged receipts for real deposits (Finding 4).
Arbitrary token contracts / recipients	Asset contracts / external users	Escrow can receive and (via inherited helpers) move arbitrary ERC721s; minted/released NFTs go to caller-chosen recipients.	Token behavior is standard and recipients can receive ERC721s.	Nonstandard tokens or malicious recipient callbacks can complicate custody; the owner's arbitrary-from transfer power is broad (Finding 5).

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
GigaMinter	mintNft(string, address, string(9))	What this function can do Mint one School NFT for a given schoolId to a chosen recipient (normally the Escrow), forwarding the 9 metadata fields to the NFT layer. What this function cannot/should not do It should not be callable by non-owners and should not mint a schoolId that already exists. Main invariant(s) onlyOwner and nonReentrant; a schoolId maps to exactly one tokenId; reverts inside _safeMint if the schoolId was already minted. Access level Owner-only (GigaAdmin)
GigaMinter	buyNft(string, address, string(9))	What this function can do Permissionlessly mint one School NFT to a caller-chosen recipient with a caller-chosen schoolId and metadata, paying at least baseFee, then forward the entire msg.value to donationReceiver. What this function cannot/should not do It should not let a donor define a real school's identifier or metadata, mint below baseFee, or mint while paused.



Contract Name	Function	Category
GigaMinter	batchMintNft(string[],address[],string[9][])	Main invariant(s) whenBuyingNotPaused, msg.value >= baseFee, nonReentrant, full msg.value forwarded. There is no access control on the caller – this is the primary attack surface (Finding 1). Access level Public payable, permissionless. What this function can do Owner mints many School NFTs in one call to length-matched recipients. What this function cannot/should not do It should not proceed with mismatched array lengths or a duplicate schoolId. Main invariant(s) onlyOwner and nonReentrant; schoolIds, recipients, and values lengths are equal; each schoolId is minted at most once. Access level Owner-only.
GigaMinter	setBaseFee(uint256)	What this function can do Update the base fee required by buyNft. What this function cannot/should not do It should not be set by non-owners; arguably it should not be zero when a fee is required. Main invariant(s) onlyOwner; there is no positivity guard, so a zero value is accepted (Finding 3). Access level Owner-only.

Contract Name	Function	Category
GigaMinter	setDonationReceiver(address payable)	What this function can do Update the address that receives all buyNft funds. What this function cannot/should not do It should not be set by non-owners or to the zero address. Main invariant(s) onlyOwner; there is no zero-address guard (Finding 2). Access level Owner-only.
GigaMinter	updateBuyingStatus(bool)	What this function can do Pause or unpause the permissionless buyNft path and emit BuyingStatusUpdated. What this function cannot/should not do It should not be toggleable by non-owners and does not gate admin minting. Main invariant(s) onlyOwner; sets isBuyingPaused to the exact value; mintNft/batchMintNft remain callable while paused. Access level Owner-only.



Contract Name	Function	Category
GigaMinter	multicall(bytes[])	What this function can do Batch multiple calls to this contract in one transaction via delegatecall-to-self. What this function cannot/should not do It should not forward value to payable functions and cannot escalate privileges. Main invariant(s) Non-payable (OZ 5.3.0), so nested buyNft carries msg.value 0; each sub-call runs under its own modifiers; the nonReentrant guard is preserved. Access level Public.
GigaMinter	transferOwnership / acceptOwnership / renounceOwnership	What this function can do Two-step ownership transfer, acceptance, and renouncement (Ownable2Step). What this function cannot/should not do It should not hand ownership to an address that has not accepted, and renounce should be deliberate. Main invariant(s) transferOwnership and renounceOwnership are onlyOwner; acceptOwnership succeeds only for the pendingOwner. Access level Owner-only (acceptOwnership: pendingOwner).

Contract Name	Function	Category
GigaMinter	Public getters (baseFee, donationReceiver, totalDonations, totalBaseFeeCollected, isBuyingPaused, schoolNft, owner, pendingOwner)	What this function can do Expose the current configuration and cumulative totals. What this function cannot/should not do They should not mutate state. Main invariant(s) Read-only mirrors of stored state; totals are informational and never decrease. Access level Public view.
Escrow	onERC721Received(address, address,uint256,bytes)	What this function can do Acknowledge an inbound ERC721 safe transfer by returning the receiver selector and emitting ERC721Received. What this function cannot/should not do It should not be treated as an authenticated deposit and ideally should verify msg.sender == nftAddress. Main invariant(s) Returns the IERC721Receiver selector unconditionally; performs no sender check, so any caller and any collection are accepted (Finding 4). Access level Public.



Contract Name	Function	Category
Escrow	receive()	What this function can do Accept native ETH into the escrow and emit Deposit. What this function cannot/should not do It should not authenticate the sender or change ownership/custody state. Main invariant(s) Received ETH becomes owner-withdrawable escrow balance. Access level Public payable.
Escrow	withdraw(address payable,uint256)	What this function can do Send a chosen amount of ETH from the escrow to a recipient. What this function cannot/should not do It should not be callable by non-owners. Main invariant(s) onlyOwner; checks the low-level call success; unlike the NFT transfers it is not marked nonReentrant (Finding 7). Access level Owner-only.

Contract Name	Function	Category
Escrow	bulkTransferSchoolNft(address[],uint256[])	What this function can do Release many School NFTs to matched recipients in one call. What this function cannot/should not do It should not proceed on mismatched array lengths. Main invariant(s) onlyOwner and nonReentrant; _tos and _tokenIds lengths are equal. Access level Owner-only.
Escrow	transferERC721 / approveERC721 / approveERC721ForAll (inherited)	What this function can do Owner moves or approves any ERC721 token, on any collection, from an arbitrary from address. What this function cannot/should not do It should not exceed the escrow's stated purpose of holding and releasing School NFTs. Main invariant(s) onlyOwner; the _from parameter is arbitrary (requires that address to have approved the escrow) – broad owner power (Finding 5). Access level Owner-only.



Contract Name	Function	Category
Escrow	transferOwnership / acceptOwnership / renounceOwnership	What this function can do Two-step ownership transfer, acceptance, and renouncement (Ownable2Step). What this function cannot/should not do It should not hand ownership to an address that has not accepted. Main invariant(s) transferOwnership and renounceOwnership are onlyOwner; acceptOwnership only by pendingOwner; renouncing permanently bricks withdraw and NFT release while assets are custodied. Access level Owner-only (acceptOwnership: pendingOwner).
Escrow	nftAddress()	What this function can do Expose the configured School NFT contract address. What this function cannot/should not do It should not change; there is no setter. Main invariant(s) Returns the address set once in the constructor. Access level Public view



3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
Escrow	Native currency and ERC721 (School NFTs, plus any ERC721 sent to it)	School NFTs minted to or transferred into the escrow and held until the owner releases them; ETH received via receive().
GigaMinter	Native currency (transient only)	Holds no assets persistently; within buyNft the entire msg.value is forwarded to donationReceiver in the same call, so the contract keeps nothing.



3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
GigaMinter	buyNft(...)	msg.value (base fee + donation).	The entire msg.value is forwarded to donationReceiver.	Any account (permissionless).	Permissionless mint with caller-chosen schoolId/metadata (Finding 1); a zero baseFee makes it free (Finding 3); a zero donationReceiver burns the ETH (Finding 2).
GigaMinter	mintNft(...) / batchMintNft(...)	None	None (no ETH); a School NFT is minted to the recipient.	Owner	Reverts if the schoolId was already minted (e.g. squatted via buyNft).
Escrow	receive()	Native ETH from any sender.	None.	Any account or contract.	ETH becomes owner-withdrawable; no per-user accounting.
Escrow	withdraw(..)	None.	None.	Owner	Owner-only; the raw call is not nonReentrant.
Escrow	transferSchoolNft(...) / bulkTransferSchoolNft(...)	None.	None.	Owner	Push-only delivery; recipient onERC721Received callback fires (guarded by nonReentrant).



Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Escrow	onERC721Received(..)	An ERC721 enters the escrow via safeTransferFrom.	None.	Any (the token contract, or a direct caller).	Accepts any collection and returns the selector unconditionally; direct calls can emit spoofed receipts.
Escrow	transferERC721(...) (inherited)	None.	Any ERC721 leaves from an arbitrary from address to a recipient.	Owner	Broad owner power over any collection and any approver (Finding 5).

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Closing Summary

In this report, we have considered the security of Gigablocks. We performed our audit according to the procedure described above.

1 High, 3 Low and 2 Info severity issues were found. Which the GigaBlocks Team has Fixed and Acknowledged.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1500+

Projects Secured

Follow Our Journey



AUDIT REPORT

July 2026

For



unicef 
for every child



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com